UNIVERSITY OF SCIENCE, VNU-HCM

Faculty of Information Technology

CSC14003 - Introduction to Artificial Intelligence

Project 2: Logic

Futoshiki Puzzles

Instructor: Nguyễn Thanh Tình

Group Members

Nguyễn Khánh Toàn	MSSV: 24127252
Nguyễn Chí Tài	MSSV: 24127529
Nguyễn Ngọc Thiên	MSSV: 24127545
Phan Quang Tiến	MSSV: 24127250

Semester II (2025 - 2026)

Contents

1 Introduction **1**
- 1.1 Project Overview . . . 1
- 1.2 Objectives and Scope . . . 1
- 1.3 Team Information . . . 1

2 Project Planning and Task Distribution **2**
- 2.1 Work Breakdown . . . 2
- 2.2 Member Responsibilities . . . 3
- 2.3 Completion Percentage per Member . . . 3
- 2.4 Project Management Board . . . 3

3 Problem Formulation **5**
- 3.1 Puzzle Rules . . . 5
- 3.2 First-Order Logic Formalization . . . 6
  - 3.2.1 Predicate Symbols . . . 6
  - 3.2.2 Core Axioms . . . 6
  - 3.2.3 Step-by-step CNF Derivation . . . 7
- 3.3 Forward Chaining (Maintaining Arc Consistency) . . . 9
  - 3.3.1 Idea and Architecture . . . 9
  - 3.3.2 Algorithm Analysis . . . 10
  - 3.3.3 Pseudocode . . . 13
  - 3.3.4 Illustration . . . 15
- 3.4 Backward Chaining (SLD Resolution) . . . 16
  - 3.4.1 Idea . . . 16
  - 3.4.2 Algorithm Analysis . . . 17
  - 3.4.3 Pseudocode . . . 20
  - 3.4.4 Illustration . . . 23
- 3.5 A* Search . . . 23
  - 3.5.1 Idea . . . 23
  - 3.5.2 Heuristic Function and Admissibility Proof . . . 24
  - 3.5.3 Pseudocode . . . 25
  - 3.5.4 Illustration . . . 26
  - 3.5.5 Algorithm Analysis . . . 26

3.6 Brute-force and Backtracking 27
3.6.1 Idea 27
3.6.2 Pseudocode 27
3.6.3 Illustration 28
3.6.4 Algorithm Analysis 28
3.7 SAT Solver (Using SAT Library) 29
3.7.1 Idea 29
3.7.2 Constraint Encoding 29
3.7.3 Pseudocode 30
3.7.4 Illustration 30
3.7.5 Algorithm Analysis 30
3.8 GUI 31
3.8.1 Overview 31
3.8.2 Screen Flow 31
3.8.3 Gameplay Interface 31
3.8.4 AI Solver Visualization 34
3.8.5 Visual Design 36

4 Experiments and Evaluation 36
4.1 Experimental Setup 36
4.2 Evaluation Protocol and Metrics 36
4.2.1 Input and Output Test Cases 36
4.2.2 Evaluation Metrics 36
4.3 Experimental Results 37
4.3.1 Aggregate Statistics (Mean/Median by Solver) 37
4.3.2 Runtime Analysis 37
4.3.3 Memory Usage Analysis 38
4.3.4 Algorithmic Complexity (Nodes Expanded) 39
4.4 Comparative Analysis 39
4.4.1 Performance Leader 39
4.4.2 Preferred Conditions 40
4.4.3 Trade-off Analysis (Implementation Complexity vs Practical Performance) 40
4.5 Self-evaluation Against Project Criteria 41

5 Conclusion 42

6 References **42**

A Appendix **42**

A.1 Generative AI Usage Disclosure 42

A.2 Project Deliverables 43

List of Tables

1 Project work breakdown aligned with assignment requirements 2
2 Member responsibilities 3
3 Completion percentage per member 3
4 Project timeline and progress board summary 5
5 Aggregate performance statistics by solver (excluding `SKIPPED` rows). 37

List of Figures

1 GUI screen flow from menu to gameplay. 31
2 Main gameplay interface in manual mode. 33
3 Step-by-step AI solver visualization on the game screen. 35
4 Runtime comparison across inputs (log-scale y-axis). 38
5 Memory vs runtime trade-off by solver. 39
6 Normalized multi-metric comparison (higher area indicates better overall balance). 41

1. Introduction

1.1. Project Overview

This project builds a full pipeline for solving *Futoshiki* puzzles in CSC14003 — Introduction to Artificial Intelligence. A Futoshiki puzzle is an $N \times N$ grid where each row/column must contain $1..N$ exactly once and all adjacency inequalities must hold.

The system is modular: I/O, domain model, knowledge base, solver modules, and GUI. It supports loading multiple inputs, running selected methods, and exporting standardized outputs. This report presents all five implemented solver families: Forward Chaining, Backward Chaining, A* Search, Brute-force/Backtracking, and SAT solving. The PySAT-based solver acts as a reliable validator to confirm the logical integrity of our core algorithms and test instances.

1.2. Objectives and Scope

The main objectives of this project are:

- Model Futoshiki constraints consistently for search-based and logic-based solving.
- Implement and evaluate AI solvers with modular, reusable code.
- Provide a practical GUI for puzzles of different sizes.
- Compare methods by correctness and efficiency.

Scope of this report:

- Present the overall project context.
- Describe all implemented solvers in detail: **Forward Chaining**, **Backward Chaining**, **A* Search**, **Brute-force/Backtracking**, and **SAT Solver (using a SAT library)**.
- Compare these methods through a unified experiment and GUI workflow.

1.3. Team Information

The project was completed by four students from the Faculty of Information Technology, University of Science — VNU-HCM:

- Nguyễn Khánh Toàn — MSSV: 24127252

- Nguyễn Chí Tài — MSSV: 24127529
- Nguyễn Ngọc Thiên — MSSV: 24127545
- Phan Quang Tiến — MSSV: 24127250

The team collaborated on modeling, implementation, testing, and reporting; detailed assignments are listed in the planning section.

2. Project Planning and Task Distribution

2.1. Work Breakdown

To align with the grading rubric, the project was divided into work packages covering FOL modeling, automatic KB generation, inference/search solvers, experiments, and final deliverables. Table 1 summarizes tasks and outputs.

Table 1: Project work breakdown aligned with assignment requirements

WP	**Major Task**	**Expected Deliverable**
WP1	FOL formalization for Futoshiki	Full axiom set (domain, uniqueness, inequality, clues) with representative Skolemization and CNF derivations.
WP2	Automatic ground KB/CNF generation	Module to instantiate constraints for any N and output solver-ready clauses/facts.
WP3	Forward Chaining solver	From-scratch rule-based engine for value inference and contradiction detection.
WP4	Backward Chaining solver	Goal-driven SLD-style querying to infer `Val(i,j,v)` facts.
WP5	A* search solver	Heuristic search with admissible evaluation and consistency pruning.
WP6	Brute-force and Backtracking baselines	Baseline solvers for correctness checks and performance comparison.
WP7	SAT-based solving support	SAT encoding and solving pipeline for verification and comparison.
WP8	Experimental setup and I/O pipeline	Standardized I/O, benchmark script, and solved-output export for required test cases.
WP9	Technical report and video demo	Final HCMUS-format report and public demo video on representative puzzles.

2.2. Member Responsibilities

Table 2: Member responsibilities

Member	**Primary Responsibilities**
Nguyễn Khánh Toàn	A* implementation; SAT integration; benchmark/export pipeline; repository coordination; README and demo coordination; GUI integration support; co-editing final report.
Nguyễn Chí Tài	Forward Chaining implementation; test-input preparation; support for report editing and validation tests.
Nguyễn Ngọc Thiên	FOL axiom design and CNF workflow; Backward Chaining implementation; support for report editing and consistency checks.
Phan Quang Tiến	Automatic KB generation from grounding/CNF logic; Brute-force and Backtracking implementations; support for report editing and comparative analysis.

2.3. Completion Percentage per Member

For grading transparency, the team uses a unified completion declaration. In this submission, each member is recorded at **100%** completion.

Table 3: Completion percentage per member

Member	**Completion Percentage**
Nguyễn Khánh Toàn	100%
Nguyễn Chí Tài	100%
Nguyễn Ngọc Thiên	100%
Phan Quang Tiến	100%

2.4. Project Management Board

The team used **GitHub** for branch-based collaboration, pull/merge workflow, and version traceability. Planning/tracking were managed on a shared board with status labels (*To do*, *In progress*, *Completed*), assignees, priorities, and deadlines.

The board followed project milestones: formalization, solver implementation, then benchmarking and reporting. This reduced late rework and prioritized rubric-weighted items.

Table 4: Project timeline and progress board summary

Period	**Stage**	**Main Activities**	**Outputs**	**Status**
08/04/2026–11/04/2026	Planning and formalization	Requirement analysis, role assignment, FOL specification, initial CNF workflow.	Work plan, FOL draft, initial puzzle inputs.	Completed
12/04/2026–17/04/2026	Core algorithm implementation	Forward/Backward Chaining, A*, SAT pipeline, KB generation, backtracking baselines.	Solver modules and intermediate validation runs.	Completed
18/04/2026–20/04/2026	Integration and benchmark	Unified I/O, solved-output export, runtime comparison, GUI connection and stability checks.	End-to-end executable system and benchmark artifacts.	Completed
21/04/2026–25/04/2026	Documentation and delivery	Report refinement, figure/table completion, demo finalization, submission packaging.	Final report PDF, demo link, and full submission package.	Completed

3. Problem Formulation

3.1. Puzzle Rules

Futoshiki is a square-grid logic puzzle played on an $N \times N$ board. The primary objective is to fill the grid with digits such that each row and each column contains every number from 1 to N exactly once. The puzzle is governed by three fundamental categories of constraints:

1. **Latin Square Property:** Similar to Sudoku, every row and every column must be a permutation of the set $\{1, 2, \ldots, N\}$. This implies

that no digit can be repeated within the same row or the same column.

2. **Inequality Constraints:** The defining characteristic of Futoshiki is the presence of "less than" (<) and "greater than" (>) signs between adjacent cells. These signs may be placed horizontally (between cells in the same row) or vertically (between cells in the same column). Any numerical assignment must strictly satisfy these relational constraints.
3. **Pre-filled Clues (Fixed Values):** At the start of the puzzle, some cells may already contain fixed digits (referred to as "Given Clues"). These values are immutable and serve as the starting points for the logical deduction process.

A puzzle is considered solved when every cell is filled with a valid digit and all structural, permutation, and inequality constraints are simultaneously satisfied. For an $N \times N$ board, the total number of cells to be determined is N^2, leading to a search space that grows factorially with N.

3.2. First-Order Logic Formalization

3.2.1 Predicate Symbols

The following vocabulary is used to formalize the Futoshiki puzzle rules into First-Order Logic sentences:

- $Val(i, j, v)$: Cell (i, j) is assigned value v
- $Given(i, j, v)$: Cell (i, j) has the pre-filled clue value v
- $LessH(i, j)$: Horizontal constraint indicating cell $(i, j) < (i, j + 1)$.
- $GreaterH(i, j)$: Horizontal constraint indicating cell $(i, j) > (i, j + 1)$.
- $LessV(i, j)$: Vertical constraint indicating cell $(i, j) < (i + 1, j)$.
- $GreaterV(i, j)$: Vertical constraint indicating cell $(i, j) > (i + 1, j)$.
- $Less(v_1, v_2)$: Integer $v1 < v2$ (background arithmetic relation)

3.2.2 Core Axioms

The logic of a Futoshiki puzzle of size NxN is encoded using the following universal axioms:

1. **Cell Completeness and Uniqueness:** Each cell must contain exactly one value.
   - $\forall i \forall j \exists v \; Val(i, j, v)$
   - $\forall i \forall j \forall v_1 \forall v_2 (Val(i, j, v_1) \wedge Val(i, j, v_2) \Rightarrow v_1 = v_2)$

2. **Value Domain and Boundary Constraints:** Values must be within the range $[1, N]$.
   - $\forall i \forall j \forall v (Val(i, j, v) \Rightarrow Less(0, v) \wedge Less(v, N + 1))$
3. **Permutation Constraints (Row and Column Uniqueness):** Every number appears exactly once in each row and column.
   - Row: $\forall i \forall j_1 \forall j_2 \forall v (Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2 \Rightarrow \perp)$
   - Column: $\forall i_1 \forall i_2 \forall j \forall v (Val(i_1, j, v) \wedge Val(i_2, j, v) \wedge i_1 \neq i_2 \Rightarrow \perp)$
4. **Inequality Constraints:** Satisfaction of all horizontal and vertical relational signs.
   - $\forall i \forall j \forall v_1 \forall v_2 (LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2) \Rightarrow Less(v_1, v_2))$
   - $\forall i \forall j \forall v_1 \forall v_2 (GreaterH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2) \Rightarrow Less(v_2, v_1))$
   - $\forall i \forall j \forall v_1 \forall v_2 (LessV(i, j) \wedge Val(i, j, v_1) \wedge Val(i + 1, j, v_2) \Rightarrow Less(v_1, v_2))$
   - $\forall i \forall j \forall v_1 \forall v_2 (GreaterV(i, j) \wedge Val(i, j, v_1) \wedge Val(i + 1, j, v_2) \Rightarrow Less(v_2, v_1))$
5. **Given Clues Enforcement:** Pre-filled digits must be assigned to their respective cells.
   - $\forall i \forall j \forall v (Given(i, j, v) \Rightarrow Val(i, j, v))$
6. **Background Arithmetic (Ordering Properties)** This axiom defines the strict ordering properties of the *Less* predicate:
   - **Irreflexivity:** $\forall v \neg Less(v, v)$
   - **Transitivity:** $\forall v_1 \forall v_2 \forall v_3 (Less(v_1, v_2) \wedge Less(v_2, v_3) \Rightarrow Less(v_1, v_3))$

   Note: For practical implementation, the Knowledge Base is populated with ground facts $Less(a, b)$ for all constant pairs (a, b) where a is numerically smaller than b within the domain $[1, N]$.

3.2.3 Step-by-step CNF Derivation

The following examples demonstrate the conversion of FOL axioms into Conjunctive Normal Form (CNF) for use in inference engines.

Example 1: Cell Value Uniqueness
FOL Form: $\forall i \forall j \forall v_1 \forall v_2 (Val(i, j, v_1) \wedge Val(i, j, v_2) \Rightarrow v_1 = v_2)$

1. **Step 1 (Eliminate Implication):** Apply the rule $A \Rightarrow B \equiv \neg A \vee B$:

$$\forall i \forall j \forall v_1 \forall v_2 (\neg(Val(i,j,v_1) \wedge Val(i,j,v_2)) \vee (v_1 = v_2))$$

2. **Step 2 (Move Negation Inward):** Apply De Morgan's Law to distribute the negation:

$$\forall i \forall j \forall v_1 \forall v_2 (\neg Val(i,j,v_1) \vee \neg Val(i,j,v_2) \vee (v_1 = v_2))$$

3. **Step 3 (Drop universal quantifiers):** Academic standard

$$\neg Val(i,j,v_1) \vee \neg Val(i,j,v_2) \vee (v_1 = v_2)$$

4. **Step 4 (Standardization for Solver) (Practical):** Since we only consider pairs where $v_1 \neq v_2$ during the grounding process, the literal $(v_1 = v_2)$ evaluates to False ($\perp$) and is removed. **CNF Clause:**

$$[\neg Val(i,j,v_1), \neg Val(i,j,v_2)] \quad \forall v_1 \neq v_2$$

Example 2: Row Uniqueness (Binary Clauses)
FOL Form: $\forall i \forall j_1 \forall j_2 \forall v (Val(i,j_1,v) \wedge Val(i,j_2,v) \wedge j_1 \neq j_2 \Rightarrow \perp)$

1. **Step 1 (Eliminate Implication):**

$$\forall i \forall j_1 \forall j_2 \forall v (\neg(Val(i,j_1,v) \wedge Val(i,j_2,v) \wedge j_1 \neq j_2) \vee \perp)$$

2. **Step 2 (Move Negation Inward):** Apply De Morgan's Law and remove the contradiction constant $\perp$:

$$\forall i \forall j_1 \forall j_2 \forall v (\neg Val(i,j_1,v) \vee \neg Val(i,j_2,v) \vee (j_1 = j_2))$$

3. **Step 3 (Drop universal quantifiers):** Academic standard

$$\neg Val(i,j_1,v) \vee \neg Val(i,j_2,v) \vee (j_1 = j_2)$$

4. **Step 4 (Refine Difference Constraint) (Practical):** Since the axiom iterates over distinct column indices, this literal is False and is

discarded during grounding. **CNF Clause:**

$$[\neg Val(i, j_1, v), \neg Val(i, j_2, v)] \quad \forall j_1 \neq j_2$$

Example 3: Horizontal Less-than
FOL Form: $\forall i \forall j \forall v_1 \forall v_2 (LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2) \Rightarrow Less(v_1, v_2))$

1. **Step 1 (Eliminate Implication):**

$$\forall i \forall j \forall v_1 \forall v_2 (\neg(LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j+1, v_2)) \vee Less(v_1, v_2))$$

2. **Step 2 (Move Negation Inward):** Apply De Morgan's Law to distribute the negation across the conjunction:

$$\forall i \forall j \forall v_1 \forall v_2 (\neg LessH(i, j) \vee \neg Val(i, j, v_1) \vee \neg Val(i, j+1, v_2) \vee Less(v_1, v_2))$$

3. **Step 3 (Drop Universal Quantifiers):** Remove the quantifiers to obtain the final clausal form: **CNF Clause:**

$$[\neg LessH(i, j), \neg Val(i, j, v_1), \neg Val(i, j + 1, v_2), Less(v_1, v_2)]$$

3.3. Forward Chaining (Maintaining Arc Consistency)

3.3.1 Idea and Architecture

Why MAC instead of pure Forward Chaining or DPLL
Pure Forward Chaining (FC) starts from known facts and applies Modus Ponens exhaustively until no new fact can be derived. For Futoshiki puzzles with few given clues, FC alone terminates with an *incomplete* assignment: many cells still have multiple candidate values and no rule fires to resolve them. The solver is stuck without guessing.

DPLL (Davis–Putnam–Logemann–Loveland), on the other hand, operates on a *propositional CNF* encoding. Every cell-value pair becomes a Boolean variable; the KB is a flat list of clauses. Unit Propagation plays the role of Forward Chaining at the Boolean level. While correct, the CNF representation discards the structure of the problem: a "domain" is implicit across N Boolean variables per cell and must be reconstructed during

branching.

MAC (Maintaining Arc Consistency) retains the native *domain* representation — each cell (i, j) holds a set of candidate values $D_{i,j} \subseteq \{1, \ldots, N\}$ — and interleaves FC-style propagation with backtracking search. Every time the search makes a tentative assignment, FC is re-run on the updated domain to prune inconsistent values from neighbouring cells before recursing deeper. This *maintains* arc consistency throughout the search, detecting dead ends far earlier than either pure FC or DPLL.

Concretely, the implementation is organised in two tiers:

- **Inference engine (`_run_fc`):** A FOL Forward Chaining engine that processes an agenda of typed facts $\{\texttt{Assign}(i, j, v), \texttt{Remove}(i, j, v)\}$ and applies five Modus Ponens rules (described below) until the agenda is empty or a contradiction is detected.
- **Search controller (`_backtrack`):** When FC cannot reduce any domain further, a branching step selects the most constrained unassigned cell (MRV), guesses a value, invokes FC, and recurses. On failure, the domain snapshot is restored and the next value is tried.

The key insight is that `_run_fc` is called *both* as an initial propagation pass over the puzzle clues and as the consistency- maintenance step after every branching decision — the same code plays both roles, which is the defining characteristic of MAC.

3.3.2 Algorithm Analysis

Domain Representation

Each cell (i, j) maintains a set $D_{i,j} \subseteq \{1, \ldots, N\}$. A cell is considered *assigned* when $|D_{i,j}| = 1$. The initial state sets $D_{i,j} = \{1, \ldots, N\}$ for every cell.

Initial Facts

The puzzle instance is converted into a seed agenda of typed facts:

- $\text{Given}(i, j, v) \Rightarrow \texttt{Assign}(i, j, v)$
- $\text{LessH}(i, j): \ (i, j) < (i, j{+}1) \Rightarrow \texttt{Remove}(i, j, N)$ and $\texttt{Remove}(i, j{+}1, 1)$
- $\text{GreaterH}(i, j): \ (i, j) > (i, j{+}1) \Rightarrow \texttt{Remove}(i, j, 1)$ and $\texttt{Remove}(i, j{+}1, N)$
- Similarly for vertical constraints LessV and GreaterV.

These correspond directly to the ground instances of FOL axioms A4–A9.

Five Modus Ponens Rules (FC Engine)

The FC engine applies the following rules on each fact popped from the agenda:

Rule	Trigger	Effect
R1	`Assign`(i, j, v) arrived, $v \notin D_{i,j}$	Contradiction $\Rightarrow$ return *False*
R2	`Remove`(i, j, v) makes $D_{i,j} = \emptyset$	Contradiction $\Rightarrow$ return *False*
R3	`Assign`(i, j, v)	Push `Remove`(i, c, v) for all $c \neq j$ (row) Push `Remove`(r, j, v) for all $r \neq i$ (column)
R4	$\lvert D_{i,j}\rvert = 1$ after a Remove	Push `Assign`(i, j, v') where v' is the sole survivor
R5	Any domain change at (i, j)	Propagate new min / max of $D_{i,j}$ to all four inequality

Rules R1–R4 correspond to the grounded FOL axioms A1–A3 (cell uniqueness and row/column permutation). Rule R5 implements A4–A9 (inequality constraints): if $(i, j) < (i, j{+}1)$, then every value $v \leq \min(D_{i,j})$ is infeasible for $(i, j{+}1)$ and is pushed as a `Remove` fact.

Backtracking with MRV

When the FC engine returns without fully assigning all cells, the search controller:

1. Selects the unassigned cell with the *minimum remaining values* (MRV) in its domain — the fail-first heuristic.
2. Takes a snapshot of the entire domain matrix.
3. Tries each value $v \in D_{i,j}$ in ascending order. For each attempt, it pushes `Assign`(i, j, v) onto a fresh agenda and calls `_run_fc`.
4. On success, recurses. On failure (FC returns *False* or the recursive call fails), restores the snapshot and tries the next v.

Complexity Analysis

Aspect	Bound	Remark
Domain size	$O(N^3)$	N^2 cells, each domain $\leq N$
FC per node	$O(N^3)$	Each value removed at most once
MRV selection	$O(N^2)$	Scan all cells; early exit at size 2
Max search depth	$\leq N^2$	One cell assigned per level
Backtracks (worst)	$O(N!)$	Full combinatorial space
Backtracks (typical)	$O(N^2)$	MRV heuristic combined with arc consistency eliminates combinatorial explosion on large inputs
Space per snapshot	$O(N^3)$	Copy of all domain sets

Advantages and Limitations

- **Stronger pruning than DPLL:** MAC propagates inequality constraints through min/max bounds on domain sets, a finer-grained operation than Boolean unit propagation. Infeasible branches are detected several levels earlier. This lightweight micro-operation allows MAC to process tens of thousands of nodes in fractions of a second, as detailed in the Experiments section.
- **Directly mirrors FOL semantics:** Facts and rules are typed (Assign / Remove) and match the FOL predicates $Val(i, j, v)$, LessH, GreaterH, etc. without a translation layer.
- **Complete:** Backtracking guarantees a solution is found whenever one exists.
- **No clause learning:** Unlike CDCL-based SAT solvers, this implementation does not record conflict clauses. Repeated conflicts in structurally similar sub-trees are re-explored.
- **Snapshot cost:** Each backtracking node copies the full $O(N^3)$ domain matrix. For $N = 9$ this is manageable (≈ 729 integers) but grows rapidly with N.

3.3.3 Pseudocode

Algorithm 1 MAC Master Coordinator

```
function SOLVE(puzzle)
    D ← initialise all domains to {1, ..., N}
    facts_0 ← InitialFacts(puzzle)          ▷ Given clues + boundary inequalities
    if not RunFC(D, facts_0) then
        return None                         ▷ Contradiction in initial facts
    end if
    if IsSolved(D) then
        return ExtractGrid(D)
    end if
    if Backtrack(D) then
        return ExtractGrid(D)
    end if
    return None
end function
```

Algorithm 2 FC Engine — Modus Ponens on Domain Facts

```
function RunFC(D, facts)
    agenda ← DEQUE(facts);  seen ← {facts};  assigned ← ∅
    while agenda is not empty do
        fact ← agenda.POPLEFT
        if fact = Assign(i, j, v) then
            if v ∉ D[i][j] then return False                ▷ R1: contradiction
            end if
            if (i, j) ∉ assigned then
                D[i][j] ← {v};  assigned.ADD((i, j))
                for c ≠ j do PUSH(Remove(i, c, v))
                end for                                     ▷ R3: row
                for r ≠ i do PUSH(Remove(r, j, v))
                end for                                     ▷ R3: column
                PropagateIneq(i, j, D, Push)                ▷ R5
            end if
        else if fact = Remove(i, j, v) then
            if v ∉ D[i][j] then continue
            end if
            D[i][j].DISCARD(v)
            if D[i][j] = ∅ then return False                ▷ R2: contradiction
            end if
            if |D[i][j]| = 1 then
                PUSH(Assign(i, j, sole value of D[i][j]))   ▷ R4
            end if
            PropagateIneq(i, j, D, Push)                    ▷ R5
        end if
    end while
    return True
end function
```

Algorithm 3 Inequality Propagation (Rule R5)

```
function PropagateIneq(i, j, D, push)
    lo ← min(D[i][j]);  hi ← max(D[i][j])
    for each neighbour (i′, j′) of (i, j) connected by an inequality do
        if (i, j) < (i′, j′) then                    ▷ (i, j) is the smaller side
            for v ∈ D[i′][j′] with v ≤ lo do
                push(Remove(i′, j′, v))
            end for
        else if (i, j) > (i′, j′) then               ▷ (i, j) is the larger side
            for v ∈ D[i′][j′] with v ≥ hi do
                push(Remove(i′, j′, v))
            end for
        end if
    end for
end function
```

Algorithm 4 MAC Backtracking with MRV

```
function Backtrack(D)
    (i, j) ← MRV(D)        ▷ Cell with smallest |D[i][j]| > 1; None if all assigned
    if (i, j) = None then return True
    end if
    for v ∈ sorted(D[i][j]) do
        D_saved ← DeepCopy(D)
        if RunFC(D, [Assign(i, j, v)]) then
            if Backtrack(D) then return True
            end if
        end if
        D ← D_saved                                  ▷ Restore snapshot
    end for
    return False
end function
```

3.3.4 Illustration

Consider a 4×4 Futoshiki puzzle with the following initial state: cell $(0, 0)$ is given as 2, cell $(3, 3)$ is given as 4, and there is a single inequality constraint $(0, 1) > (0, 2)$. The initial agenda is populated with these base facts.

1. **FC pass 1 (Initial Propagation):** The `Assign`$(0, 0, 2)$ and `Assign`$(3, 3, 4)$ facts are processed. Rule R3 pushes `Remove` facts to eliminate 2 from row 0 and column 0, and 4 from row 3 and column 3. The inequality $(0, 1) > (0, 2)$ triggers Rule R5, pushing `Remove`$(0, 1, 1)$ (as the larger cell must be strictly greater than the absolute minimum 1) and `Remove`$(0, 2, 4)$ (as the smaller cell must be strictly less than the absolute maximum 4). As the domain $D[0, 1]$ loses values 1 (due to the inequality) and 2 (due to row 0), its remaining domain becomes $\{3, 4\}$. The domain $D[0, 2]$ loses 2 and 4, narrowing down to $\{1, 3\}$.
2. **FC Stuck:** The agenda eventually becomes empty. Several cells still have $|D| > 1$. The `IsSolved` function evaluates to *False*, meaning the pure deduction phase has halted without a complete assignment.
3. **Backtracking (MRV) and Cascade:** The search controller selects the unassigned cell with the Minimum Remaining Values. Suppose it picks $(0, 1)$ with $D = \{3, 4\}$. It first guesses $v = 3$ by pushing `Assign`$(0, 1, 3)$ into a fresh agenda and invoking `RunFC`. This triggers Rule R3, pushing `Remove`$(0, 2, 3)$ to eliminate 3 from the rest of row 0. The domain $D[0, 2]$ (which was previously $\{1, 3\}$) loses 3, reducing its size to exactly 1 ($D[0, 2] = \{1\}$). Rule R4 immediately fires, pushing `Assign`$(0, 2, 1)$. Concurrently, Rule R5 propagates the new $\max(D[0, 1]) = 3$, enforcing that $(0, 2)$ must be < 3, which perfectly aligns with the new singleton domain. This single heuristic guess creates a powerful chain reaction (cascade) that rapidly collapses the remaining domains, successfully finding the solution without needing further backtracks.

The total number of backtracks on this instance is typically zero or one, because arc consistency eliminates almost all ambiguity after the first FC pass.

3.4. Backward Chaining (SLD Resolution)

3.4.1 Idea

The implemented solver is a hybrid architecture that combines Prolog-style SLD resolution with explicit state-space backtracking.

- **KB construction ($HornConverter$):** Converts a puzzle instance into Horn facts and rules using predicates such as $val(i, j, v)$, $given(i, j, v)$, and $not_val(i, j, v)$.
- **Backward chaining core (fol_bc_or / fol_bc_and):** A Prolog-like SLD interpreter with unification, substitution, DFS goal reduction, cycle detection, and NAF handling.
- **Search controller ($_backtrack$):** Controls assignment of values to cells. At each step, it selects one unassigned cell, queries $val(r, c, v)$ to obtain legal candidates, asserts one candidate as a temporary fact, recurses, and retracts on failure.

Why search is integrated with SLD Resolution: The axiom $\forall i \forall j \exists v\ Val(i, j, v)$ contains existential quantification and cannot be encoded directly as a definite Horn rule. Therefore, the solver uses backtracking for value choice, while the backward chaining engine is used to validate and prune candidates through derivable constraints (mainly not_val).

Technical Implementation Notes:

- **0-based indexing in code:** Internally, rows/columns are represented as $0..N-1$ for direct mapping to Python lists.
- **Cell-level candidate generation with NAF:** The generator rule $val(i, j, v) \leftarrow domain(v),\ naf(not_val(i, j, v))$ is used to propose candidates that are not provably forbidden.
- **Hybrid workflow:** BC serves as a logical inference engine; backtracking constructs a full solution by incrementally asserting/retracting val facts.

3.4.2 Algorithm Analysis

Atoms (Predicates)

The Horn KB used the following vocabulary:

- $val(i, j, v)$: Cell (i, j) is assigned the value v.
- $given(i, j, v)$: Cell (i, j) has the pre-filled clue value v
- $less_h(i, j)$ / $greater_h(i, j)$: Horizontal inequalities between (i, j) and $(i, j+1)$.

- $less_v(i, j)$ / $greater_v(i, j)$: Vertical inequalities between (i, j) and $(i + 1, j)$.
- $less(v1, v2)$: Background arithmetic fact representing $v_1 < v_2$.
- $neq(x, y)$: Background inequality fact representing $x \neq y$.
- $not_val(i, j, v)$: Value v is explicitly forbidden at cell (i, j).

Facts

- **Background facts**: All valid $less(v1, v2)$ and $neq(x, y)$ combinations within the finite domain $1..N$.
- **Instance facts**: $given(...)$ clues, inequality signs, and immediate "boundary pruning" facts ($not_val(...)$) derived directly from extreme inequality conditions (e.g., a cell on the smaller side of an inequality cannot hold the maximum value N).

Rules (Definite Horn)

Constraints are encoded by Horn rules that derive either val (from givens) or not_val (forbidden assignments):

- Given $\Rightarrow Val : val(i, j, v) : -given(i, j, v)$
- Cell uniqueness: If a value is chosen, all others are forbidden in that cell.
- Row/Col uniqueness: If $val(i, j1, v)$ holds, then $not_val(i, j2, v)$ for all $j_2 \neq j_1$.
- Inequalities: Values that violate a $<$ or $>$ constraint trigger a not_val derivation.

Existence / Choice Rule via NAF

To generate candidates for an empty cell, the solver adds:

$$val(i, j, v) \leftarrow domain(v),\ naf(not_val(i, j, v)).$$

A value v is accepted as a candidate for (i, j) when v is in the domain and $not_val(i, j, v)$ cannot be proven.

Unification and Substitution

- $unify(x, y, \theta)$: Computes the Most General Unifier (MGU) for two expressions, returning the substitution dictionary θ or "failure".

- $subst(\theta, expr)$: Applies the substitution θ to goals and rule bodies. Variables are represented as lowercase strings (e.g., 'v', 'i', 'j').

SLD Resolution (AND/OR Search Tree)

- OR node (bc_or): Proves a goal by attempting to match and unify it against the head of any available fact or rule.
- AND node (bc_and): Proves the body of a rule by proving all subgoals sequentially from left to right.
- Evaluation follows depth-first, leftmost goal expansion.

Negation as Failure (NAF) Handling

- $naf(G)$ succeeds iff the engine fails to derive G.
- Inside NAF context ($in_naf = True$), rule expansion for predicate val is disabled. This prevents self-supporting loops between val and $naf(not_val)$.

Optimizations Implemented

- **MRV variable ordering:** Backtracking chooses the unassigned cell with minimum remaining values (MRV) based on current BC query results.
- **Predicate indexing:** Facts and rules are indexed by predicate ($fact_index$, $pred_rule_index$, $exact_rule_index$).
- **Cell-specific index for val:** Additional indexes by cell (i, j) are used for both facts and rules of predicate val, reducing scan cost for queries of the form $val(r, c, v)$.
- **Versioned memoization:** Memo keys include $kb_version$, so cache is reused across queries while remaining correct after assert/retract operations.
- Cycle Detection (visited set): Tracks the active resolution path to prevent infinite recursion in cyclic goal graphs.
- Boundary Pruning: Converting extreme inequalities into immediate not_val facts drastically reduces the search space before resolution even begins.

3.4.3 Pseudocode

Algorithm 5 Master Coordinator

```
function SOLVE(puzzle)
    kb ← HornConverter(puzzle).build()
    add facts domain(1..N)
    add facts val(r, c, v) for all givens
    add generator rules val(i, j, v) : −domain(v), naf(not_val(i, j, v))
    build fact/rule indexes and cell indexes for val
    initialize kb_version ← 1
    for each given (r, c) = v do
        if FOL_BC_ASK(kb, not_val(r, c, v)) succeeds then
            return failure // Contradiction in givens
        end if
    end for
    return _BACKTRACK(kb)
end function
```

Algorithm 6 State Space Manager

```
function BACKTRACK(kb)
    if grid complete then
        return success
    end if
    (r, c, C) ← MRV-SelectCell(kb)
    if C = ∅ then
        return failure
    end if
    for each value in C do
        grid[r][c] ← value
        assert fact val(r, c, value) into kb
        kb_version ← kb_version + 1
        if _BACKTRACK(kb) success then
            return success
        end if
        retract fact val(r, c, value) from kb
        kb_version ← kb_version + 1
        grid[r][c] ← 0
    end for
    return failure
end function
```

Algorithm 7 Inference Interface

```
function FOL_BC_ASK(kb, goal)
    return FOL_BC_OR(kb, goal, theta = ∅, in_naf = false, visited = ∅)
end function
```

Algorithm 8 Goal Resolver - OR Node

```
function FOL_BC_OR(kb, goal, theta, in_naf, visited)
    g ← SUBST(θ, goal)
    if (g, in_naf) ∈ visited then
        return ∅                                        ▷ Cycle detection
    end if
    visited′ ← visited ∪ {(g, in_naf)}
    k ← (kb_version, g, in_naf, θ)
    if k ∈ memo then
        return memo[k]
    end if
    if g is naf(inner) then                             ▷ Negation As Failure
        if FOL_BC_OR(kb, inner, theta, true, visited′) yields no proof then
            return {theta}                              ▷ NAF success
        else
            return ∅                                    ▷ NAF failure
        end if
    end if
    answers ← ∅
    facts ← indexed facts matching pred(g)
    if pred(g) = val and (i, j) of g is grounded then
        facts ← cell-indexed facts for (i, j)
    end if
    for each fact F in facts do
        theta′ ← UNIFY(args(g), args(head(F)), theta)
        if theta′ ≠ failure then
            answers ← answers∪FOL_BC_AND(kb, body(F), theta′, in_naf, visited′)
        end if
    end for
    if not (in_naf and pred(g) = val) then
        choose rule candidates from exact/predicate/cell index
        for each rule R in candidates do
            theta′ ← UNIFY(args(g), args(head(R)), theta)
            if theta′ ≠ failure then
                answers ← answers∪FOL_BC_AND(kb, body(R), theta′, in_naf, visited′)
            end if
        end for
    end if
    memo[k] ← answers
    return answers
end function
```

Algorithm 9 Rule Body Validator - AND Node

```
function FOL_BC_AND(kb, goals, theta, in_naf, visited)
    if goals is empty then
        return {theta}
    end if
    first ← SUBST(theta, goals[0])
    rest ← goals[1 :]
    answers ← ∅
    for each theta′ in FOL_BC_OR(kb, first, theta, in_naf, visited) do
        answers ← answers∪FOL_BC_AND(kb, rest, theta′, in_naf, visited)
    end for
    return answers
end function
```

3.4.4 Illustration

On a query $val(r, c, v)$ for an unassigned cell, the resolver first tries indexed val facts for (r, c), then indexed val rules for (r, c) (plus generic generator rules). The result is a candidate set $\mathcal{C}$.

MRV chooses the cell with smallest $|\mathcal{C}|$. If a candidate $v \in \mathcal{C}$ is tentatively assigned, the solver asserts $val(r, c, v)$ into KB and increments $kb_version$. This makes old memo entries automatically inactive (because memo key includes $kb_version$), while still preserving cache reuse for repeated subgoals inside the same KB state.

If recursion fails, the fact is retracted, $kb_version$ is incremented again, and the next candidate is tried. This loop continues until a complete grid is found or the branch set is exhausted.

3.5. A* Search

3.5.1 Idea

Each state is a partial Futoshiki assignment (0 means unfilled). The solver expands states by

$$f(s) = g(s) + h(s),$$

where each action assigns one value to one empty cell (unit cost), $g(s)$ is the number of assignments already made, and $h(s)$ estimates the remaining assignments.

To reduce branching, the next cell is selected by MRV (smallest domain). Candidate values are filtered by row/column uniqueness and inequality constraints; optional AC-3 further removes unsupported values. This combines informed search (A*) with CSP-style pruning.

3.5.2 Heuristic Function and Admissibility Proof

Let $U(s)$ be the number of unassigned cells in state s. We use:

$$h(s) = U(s) = N^2 - A(s),$$

where $A(s)$ is the number of assigned cells.

AC-3 is used as a separate feasibility check: if any domain becomes empty, the state is discarded.

Admissibility:

- Any complete solution from state s must assign all $U(s)$ empty cells.
- Each action can assign at most one cell.
- Hence the true remaining cost $h^*(s)$ satisfies $h^*(s) \geq U(s) = h(s)$.

So $h(s)$ never overestimates and is admissible.

Consistency:

- For one valid move $s \to s'$, $U(s') = U(s) - 1$.
- With unit step cost $c(s, s') = 1$:

$$h(s) = U(s) = 1 + U(s') = c(s, s') + h(s').$$

- Therefore h is consistent.

3.5.3 Pseudocode

Algorithm 10 A* for Futoshiki (with MRV and AC-3)

```
Build initial state s_0 from puzzle
Validate givens; if invalid then return failure
Compute domains and heuristic for s_0
Initialize closed as an empty set
Initialize best_g[s_0]
Push s_0 into priority queue by key (f, h, −g)
while priority queue not empty do
    Pop best state s
    if s ∈ closed then
        continue
    end if
    Add s to closed
    if s is goal then return solved grid
    end if
    Select unassigned cell (r, c) by MRV
    for each value v in domain(r, c) do
        if assigning v is consistent then
            Create child state s′
            if s′ ∈ closed then
                continue
            end if
            Compute g(s′)
            if best_g[s′] exists and g(s′) is not better then
                continue
            end if
            Recompute domains (and AC-3 if enabled)
            if no empty domain in s′ then
                Set g(s′) = g(s) + 1, evaluate h(s′), f(s′) = g(s′) + h(s′)
                Update best_g[s′]
                Push s′ into priority queue
            end if
        end if
    end for
end while
return failure
```

3.5.4 Illustration

For a 4×4 instance, the solver:

1. Initializes domains for empty cells.
2. Applies consistency filtering (and AC-3 if enabled).
3. Chooses the most constrained cell by MRV.
4. Expands assignments by priority $f = g + h$.

Example progression:

- State s_0: clue cells fixed, inequalities loaded.
- State s_1: assign MRV cell, domains shrink.
- State s_2: additional forced assignment appears.
- Goal: all cells assigned; row/column uniqueness and inequalities satisfied.

3.5.5 Algorithm Analysis

Let N be board size (N^2 cells) and b the average branching factor after filtering.

Time complexity (worst case):

- Exponential search in remaining depth, approximately $O(b^d)$ with $d \leq N^2$.
- AC-3 adds polynomial overhead per state; runtime is mainly determined by expanded states.

Space complexity:

- A* stores open/closed states and domains, so memory use is higher than depth-first methods.
- Worst-case space is also exponential.

Practical behavior:

- $h(s) = U(s)$ is admissible and simple to justify.
- Most speedup comes from MRV and AC-3 pruning.
- *closed* avoids re-expansion; *best_g* removes inferior duplicate paths.
- Works well on medium instances, but memory pressure can rise on larger or weakly constrained puzzles.

3.6. Brute-force and Backtracking

3.6.1 Idea

Brute-force is a straightforward approach that tries all possible assignments for the empty cells in the Futoshiki puzzle. For a puzzle of size $N \times N$, each empty cell can take a value from 1 to N. The algorithm generates all possible combinations and checks whether they satisfy the constraints (row, column, and inequality constraints). However, this approach is highly inefficient due to the exponential number of possibilities.

Backtracking is an optimized version of brute-force. Instead of generating all possible assignments, it builds the solution step by step and abandons a partial assignment as soon as it violates any constraint. At each step, the algorithm selects an empty cell, tries a valid value, and recursively proceeds to the next cell. If a conflict occurs, it backtracks and tries another value. This significantly reduces the search space compared to brute-force.

3.6.2 Pseudocode

Brute-force:

Algorithm 11 Brute-force

```
function BRUTE_FORCE(grid):
    empty_cells = list of all empty positions
    for each possible assignment of values to empty_cells:
        fill grid with assignment
        if is_valid(grid):
            return grid
    return failure
```

Backtracking:

Algorithm 12 Backtracking

```
function BACKTRACK(grid):
    if no empty cell:
        return grid

    (row, col) = find an empty cell

    for value from 1 to N:
        if is_valid(row, col, value):
            grid[row][col] = value

            result = BACKTRACK(grid)
            if result != failure:
                return result

            grid[row][col] = empty

    return failure
```

3.6.3 Illustration

Consider a 4×4 Futoshiki puzzle with some empty cells.

Brute-force will generate all possible combinations for the empty cells. For example, if there are k empty cells, it will try up to N^k assignments, checking each one against all constraints.

Backtracking, on the other hand, fills the grid step by step. Suppose it assigns a value that violates a constraint (e.g., duplicates in a row or breaks an inequality). Instead of continuing, it immediately backtracks and tries another value. This pruning avoids exploring invalid branches of the search tree, making it much more efficient in practice.

3.6.4 Algorithm Analysis

Brute-force Complexity: The time complexity is $O(N^k)$, where k is the number of empty cells. In the worst case (empty grid), this becomes $O(N^{N^2})$, which is infeasible for large N.

Backtracking Complexity: The worst-case time complexity is still exponential, but significantly reduced in practice due to pruning. The actual

performance depends on how early invalid assignments are detected.

Space Complexity: Backtracking uses recursion, so the space complexity is $O(N^2)$ for the grid plus $O(N^2)$ for the recursion stack in the worst case.

Comparison:

- Brute-force explores all possibilities without pruning.
- Backtracking prunes invalid states early, making it much more efficient.
- Backtracking is the preferred method for solving constraint satisfaction problems like Futoshiki.

3.7. SAT Solver (Using SAT Library)

3.7.1 Idea

The SAT method encodes Futoshiki constraints into CNF and solves them with PySAT (Glucose3). In this project, `FOLKB` generates clauses, and the SAT model is decoded into an $N \times N$ solution grid.

3.7.2 Constraint Encoding

Instead of repeating full FOL derivations from Section 3.2, we summarize the SAT encoding. For each (i, j, v):

$$X_{i,j,v} = \text{true} \iff \text{cell } (i, j) \text{ has value } v.$$

Variable mapping used in code:

$$\text{ID}(i, j, v) = i \cdot N^2 + j \cdot N + v,$$

with $i, j \in [0, N-1]$, $v \in [1, N]$, so IDs are in $[1, N^3]$.

Main clause groups:

- each cell has exactly one value;
- row and column uniqueness;
- given clues as unit clauses;
- inequality constraints by forbidding violating adjacent pairs.

Any satisfying assignment corresponds to a valid puzzle solution.

3.7.3 Pseudocode

Algorithm 13 SAT-based Futoshiki Solver

```
Build CNF clauses from puzzle constraints using FOLKB
Create SAT solver instance (Glucose3)
Add all CNF clauses to solver
if solver returns UNSAT then
    return failure
end if
Get SAT model (set of literals)
Initialize empty grid N × N
for each positive literal ℓ in model do
    if 1 ≤ ℓ ≤ N^3 then
        Decode ℓ → (i, j, v)
        Set grid(i, j) ← v
    end if
end for
if any cell remains unassigned then
    return failure
else
    return solved grid
end if
```

3.7.4 Illustration

For a 4×4 puzzle:

1. Create $4^3 = 64$ variables $X_{i,j,v}$.
2. Add clauses for uniqueness, clues, and inequalities.
3. Run Glucose3 and obtain a satisfying model.
4. Decode literals to reconstruct the solved grid.

3.7.5 Algorithm Analysis

Encoding cost:

- Number of variables is N^3.
- Number of clauses is polynomial in N, dominated by exclusivity and inequality constraints.

Solving cost:

- SAT is NP-complete in general (worst-case exponential).

- In practice, CDCL solvers perform well on structured instances.

Space and implementation trade-offs:

- Requires storing CNF and SAT internal state.
- Easy to extend by adding new clause templates.

Practical behavior in this project:

- Gives clear SAT/UNSAT feasibility checking.
- Runtime is competitive in experiments, while SAT is presented as a representative method in this report.

3.8. GUI

3.8.1 Overview

The GUI is built with `pygame` for end-to-end interaction: input selection, manual play, AI solving, and result display. A modular screen/component design keeps rendering and gameplay logic maintainable.

3.8.2 Screen Flow

The interface follows a simple state flow:

- **Menu**: Play, AI Solver, Exit.
- **Selection**: choose board size and input file.
- **Game**: manual play or AI visualization.

Figure 1: GUI screen flow from menu to gameplay.

3.8.3 Gameplay Interface

The gameplay screen renders the grid, keeps clue cells fixed, and validates moves in real time. Main controls:

- click to select a cell;
- keys 1 to N to fill values;
- Backspace/Delete/0 to clear;
- timer display.

Inequality symbols are shown between cells, and completion is announced when all constraints are satisfied.

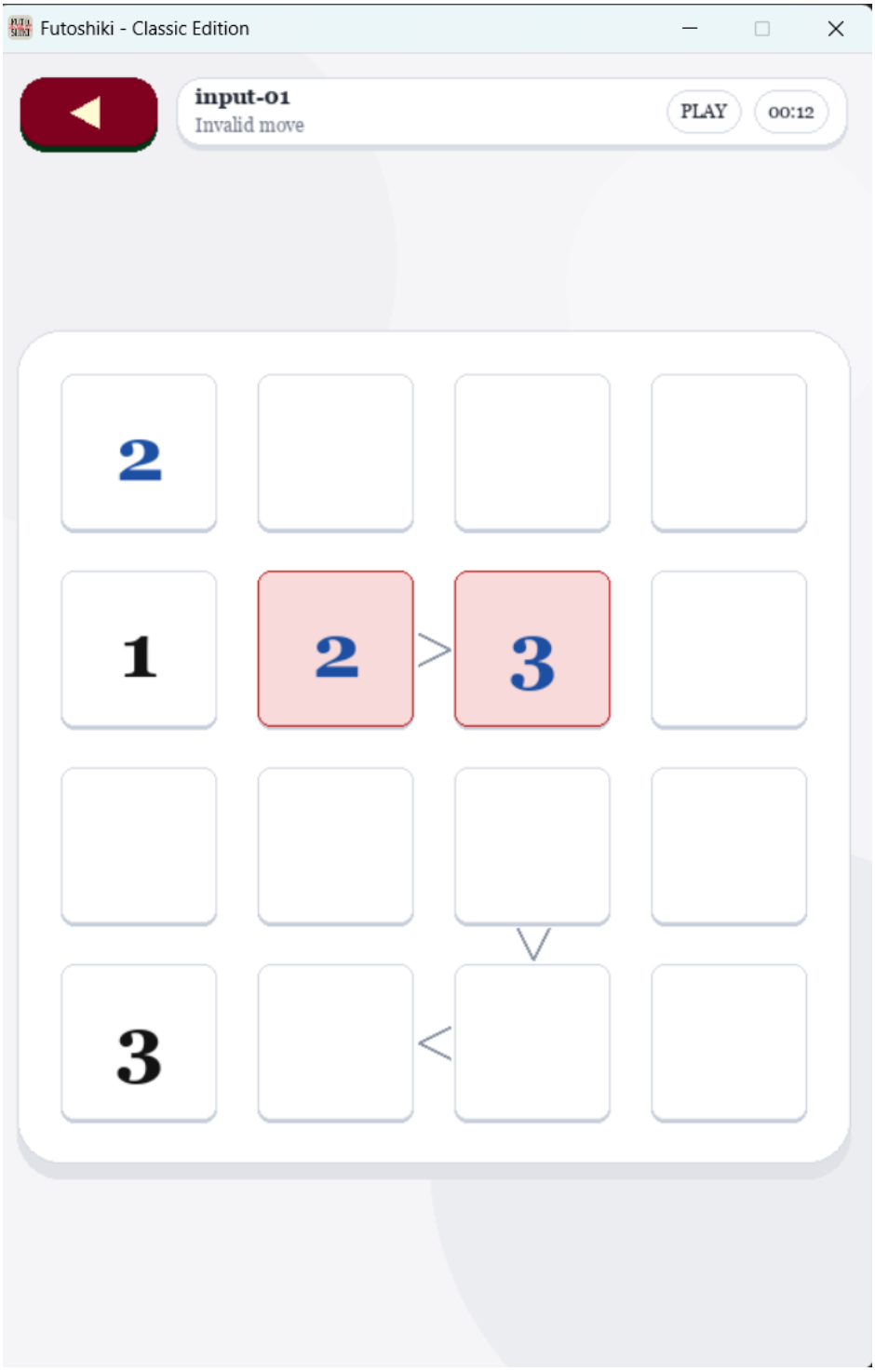

Figure 2: Main gameplay interface in manual mode.

3.8.4 AI Solver Visualization

AI visualization is handled by `AISolverManager` with a worker thread and step queue, so the UI remains responsive while solving. Solver steps are streamed incrementally to the board so users can observe the solving process.

Key behaviors:

- highlight current solver actions;
- distinguish backtracking steps;
- allow skipping remaining animation;
- validate final state before showing solved status.

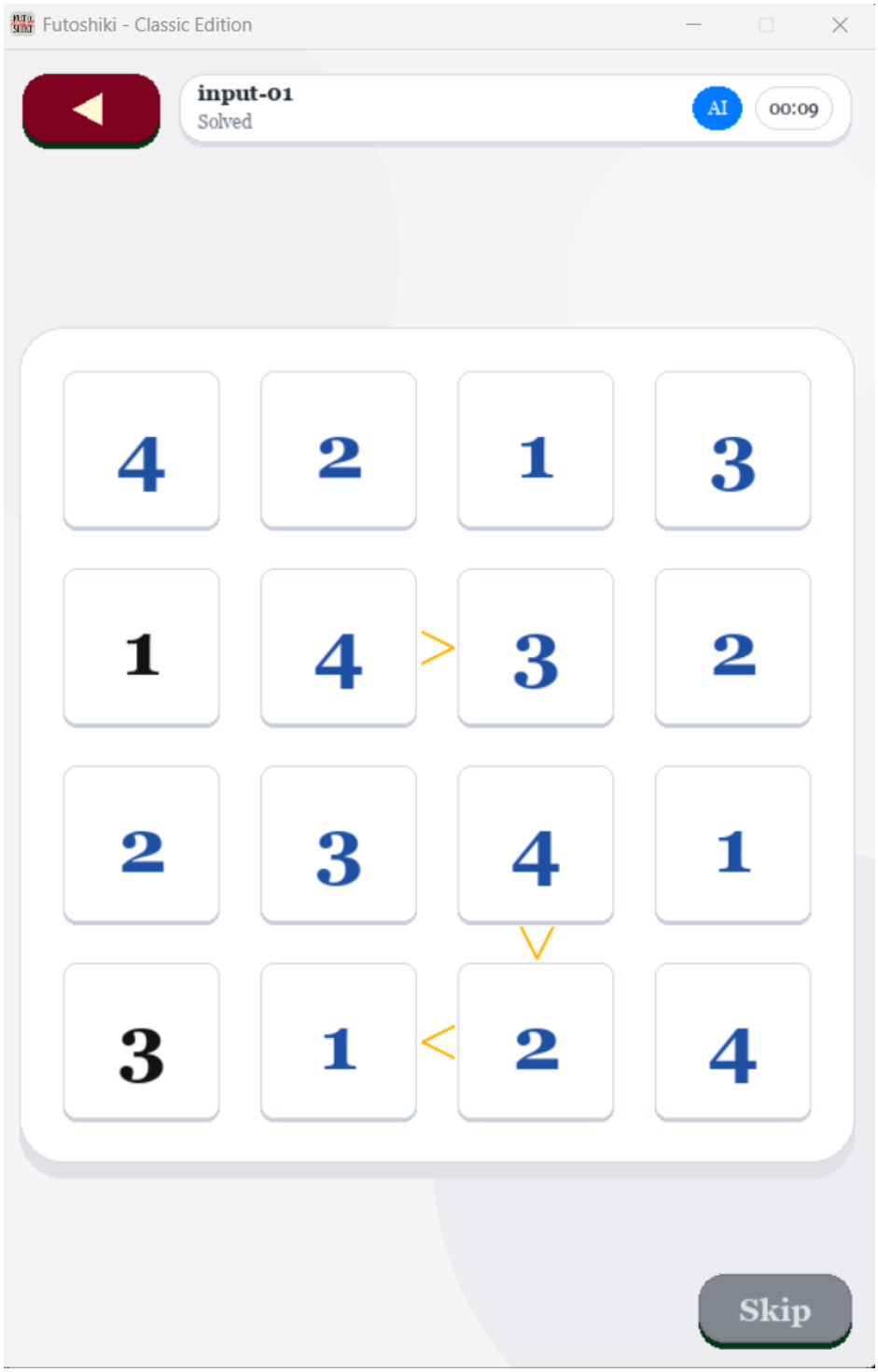

Figure 3: Step-by-step AI solver visualization on the game screen.

3.8.5 Visual Design

The visual style is simple and consistent to prioritize board readability and solver progress. Reusable UI components keep interactions uniform and implementation complexity low.

Overall, the GUI bridges puzzle data, user interaction, and solver execution in a practical way.

4. Experiments and Evaluation

4.1. Experimental Setup

We evaluate solvers on 10 instances (`input-01` to `input-10`) from 4×4 to 9×9. Results are stored in `solved.csv` and visualized by `generate_solver_graphs.py`. Main metrics are runtime (seconds), peak memory (MB via `tracemalloc`), and search effort (`node_expanded`).

For bounded and reproducible runs, each solver uses a node cap:

- `MAX_NODES_BRUTE_FORCE = 100000`
- `MAX_NODES_BACKTRACKING = 3000000`
- `MAX_NODES_BACKWARD_CHAINING = 1500000`
- `MAX_NODES_FORWARD_CHAINING = 5000000`
- `MAX_NODES_ASTAR = 5000000`

A wall-clock timeout of 180 seconds is also enforced.

4.2. Evaluation Protocol and Metrics

4.2.1 Input and Output Test Cases

The test suite satisfies the assignment requirement on size diversity. Each run returns: (i) solved grid, (ii) `False`/no solution under node cap, (iii) `time_out` at 180 s, or (iv) `SKIPPED` for intentionally excluded brute-force runs on intractable cases.

4.2.2 Evaluation Metrics

- **Runtime**: execution time in seconds.
- **Memory Usage**: peak memory in MB.

- **Algorithmic Complexity Proxy**: number of expanded/inferred nodes.

4.3. Experimental Results

4.3.1 Aggregate Statistics (Mean/Median by Solver)

Table 5 reports per-solver mean and median values. Following the plotting pipeline, `SKIPPED` rows are excluded.

Table 5: Aggregate performance statistics by solver (excluding `SKIPPED` rows).

Solver	n	Success Rate	Mean Time (s)	Median Time (s)	Mean Mem. (MB)	Median Mem. (MB)	Mean Nodes	Median Nodes
Forward Chaining	10	100%	0.027218	0.006389	0.090923	0.085709	5004.9	984.5
SAT Solver	10	100%	0.026800	0.009222	0.602992	0.275066	0	0
A* Search	10	100%	1.260518	0.301315	0.182252	0.104591	121.2	38.0
Backtracking	10	90%	6.632425	0.021260	0.002509	0.002480	324446.7	1434.0
BackwardSolver	10	90%	43.366318	2.282411	416.955660	28.340975	735.6	112.5
BruteForceSolver	2	0%	3.037844	3.037844	0.002975	0.002975	100001.0	100001.0

The mean-median gap (especially for Backtracking/Backward Chaining) indicates heavy-tail behavior on hard inputs.

4.3.2 Runtime Analysis

Figure 4 uses a log-scale y-axis. Forward Chaining is the fastest overall and typically runs in milliseconds. A* is stable and usually below 10 s. Backtracking is fast on many cases but degrades sharply on difficult inputs (about 62 s on `input-07`). Backward Chaining has the largest variance and includes one timeout (180 s).

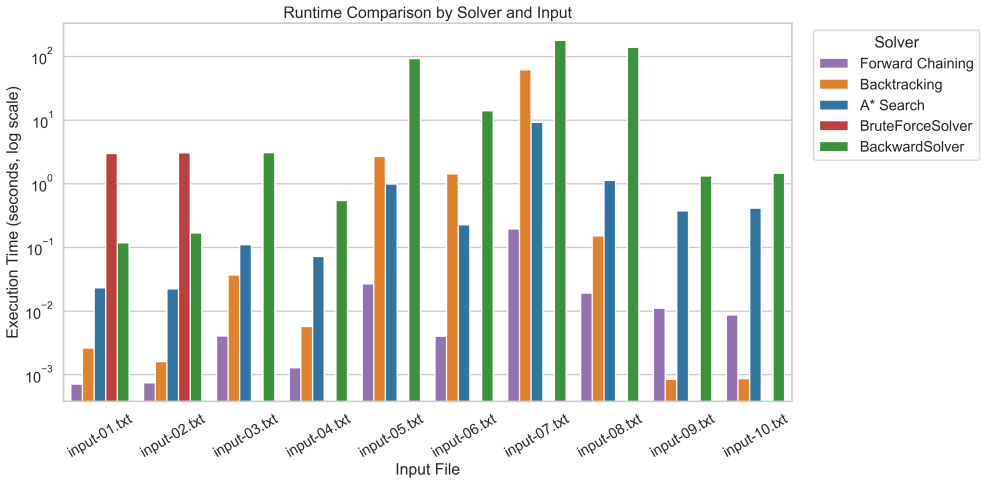

Figure 4: Runtime comparison across inputs (log-scale y-axis).

Why log scale is necessary. Runtime spans roughly 10^{-3} s to 10^{2} s. A linear axis would compress fast methods near zero and hide meaningful differences.

4.3.3 Memory Usage Analysis

Figure 5 shows that Forward Chaining, Backtracking, Brute Force, and A* are memory-efficient on this dataset, while Backward Chaining can exceed 1.8 GB near timeout.

Figure 5: Memory vs runtime trade-off by solver.

Memory explosion in recursive logic search. Backward Chaining combines recursive goal expansion, memoization, and dynamic fact/rule accumulation. On large constrained cases, many intermediate derivations are kept before contradictions are found, causing rapid memory growth.

4.3.4 Algorithmic Complexity (Nodes Expanded)

Node counts match the runtime trends. A* expands very few nodes (tens to hundreds) due to MRV + AC-3 pruning. Forward Chaining can expand more nodes but remains fast because each propagation step is lightweight. Backtracking can approach its cap (3,000,000) on hard inputs. Brute Force quickly hits its cap (100,000). Backward Chaining has moderate node counts but expensive per-node processing, explaining high runtime and memory.

4.4. Comparative Analysis

4.4.1 Performance Leader

From `solved.csv`, **Forward Chaining** is the best overall runtime performer (mean 0.027 s). **SAT Solver** is highly competitive (median 0.0092 s) and also achieves 100% success. **A*** is the next strongest practical method with 100% success and low node expansion.

Success rates on evaluated cases are: Forward Chaining 10/10, A* 10/10, SAT 10/10, Backtracking 9/10, Backward Chaining 9/10 (one timeout), Brute Force 0/2 under cap.

These results are consistent with Chapter 3: Forward Chaining benefits from strong propagation, SAT benefits from CDCL search, and A* benefits from heuristic pruning.

4.4.2 Preferred Conditions

Recommended usage:

- **Brute Force/Backtracking ($N \leq 5$)**: simple and interpretable; can degrade sharply on hard larger inputs.
- **A* Search**: suitable when heuristic guidance is available and controlled expansion is needed.
- **Forward Chaining ($N \geq 6$, low-to-medium constraint density)**: strong when early propagation reduces domains quickly.
- **SAT Solver ($N \geq 5$, varied densities)**: robust general-purpose option with strong median speed and 100% success in this benchmark.

4.4.3 Trade-off Analysis (Implementation Complexity vs Practical Performance)

Figure 6 summarizes normalized multi-metric performance. Forward Chaining and A* are balanced leaders; Backtracking is memory-light but runtime-unstable on hard cases; Backward Chaining is limited by runtime/memory growth.

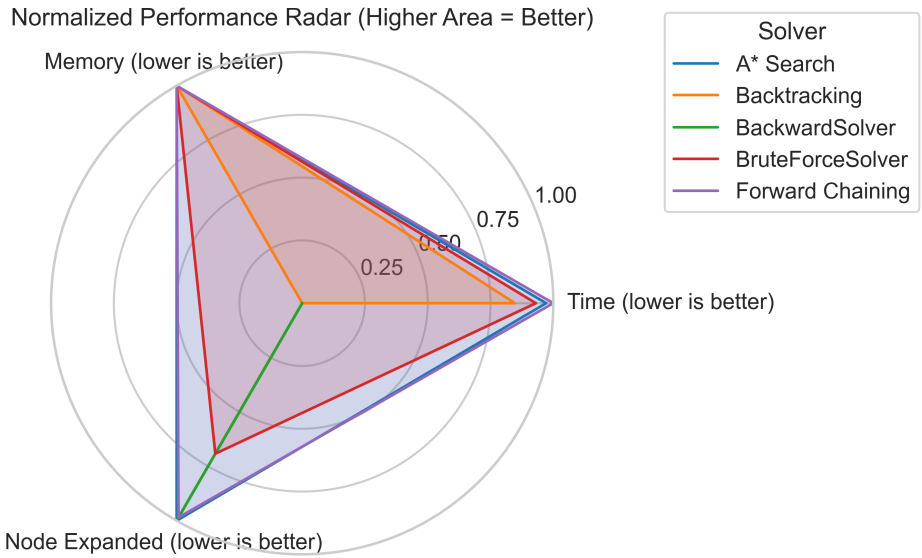

Figure 6: Normalized multi-metric comparison (higher area indicates better overall balance).

Implementation trade-offs:

- **Brute Force**: simplest, least scalable.
- **Backtracking**: moderate complexity, good on small/medium cases, unstable worst case.
- **Forward Chaining**: higher implementation effort, excellent practical balance.
- **A***: high implementation effort, very efficient expansion.
- **SAT Solver**: moderate-to-high encoding effort, strong speed and robustness.

4.5. Self-evaluation Against Project Criteria

The project satisfies core criteria: multiple solving paradigms were implemented, required test-size diversity is covered, and experiments report runtime, memory, and node expansion with visual analysis. SAT is fully integrated into the benchmark pipeline with strong empirical results. Future improvements include per-size variance analysis and deeper memory profiling on larger instances.

5. Conclusion

This project delivers a complete Futoshiki-solving system that combines multiple AI paradigms with an interactive GUI. Experiments confirm clear trade-offs among search, logic inference, and SAT in runtime, memory, and node expansion.

The codebase is organized for reproducibility and extension, with standardized I/O, solver abstractions, and automated tests. The implementation stack remains lightweight and aligned with requirements (`pygame`, `python-sat`, `pytest`).

Overall, the objectives were achieved: correct constraint modeling, implementation of representative methods, benchmark-based evaluation, and clear presentation through both CLI and GUI workflows.

6. References

- **Russell, S. J., & Norvig, P. (2020).** *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Online resource.
- **Futoshiki.com**. Reference used to collect sample input puzzles for the project.
- **Futoshiki.uk**. Reference used to compare and verify output puzzles for the project.

A. Appendix

A.1. Generative AI Usage Disclosure

This section summarizes the use of Large Language Models (LLMs) in this project:

- **AI Tools Used**: Gemini, ChatGPT, Claude.
- **Usage Scenarios**:
  - Debugging and optimizing search algorithms.
  - Formatting the LaTeX report and fixing matplotlib plots.
  - Improving English grammar in documentation.
- **Sample Prompts**:

- "How should I enforce row/column uniqueness and inequality constraints efficiently in a Futoshiki backtracking solver?"
- "In our A* Futoshiki solver, what is a good admissible heuristic to reduce node expansions on 7x7 and 9x9 inputs?"
- "Help me format a LaTeX table that compares runtime, memory, and expanded nodes across Backtracking, Forward Chaining, Backward Chaining, SAT, and A* for the 10 input files."

A.2. Project Deliverables

- **Demo Video**: Demonstration of GUI, manual gameplay, and solver animation: Click to see video.
- **Futoshiki-Puzzles Project Repository, GitHub**: Click to see repository